LG U+ Why Not SW camp

객체 탐지 기반 자율주행 차량 로봇의 **포트홀 감지 및 자동 신고 서비스**



팀 구성 및 역할



팀장

- 이름 : 김승엽
- 역할
  - 데이터 분석
  - AI 엔지니어
  - PM



팀원

- 이름 : 김나현
- 역할
  - 로봇 엔지니어
  - 클라우드 인프라
  - 클라우드 엔지니어



팀원

- 이름 : 김동혁
- 역할
  - 데이터 분석
  - 모델 엔지니어
  - 시스템 인프라 엔지니어



팀원

- 이름 : 김준희
- 역할
  - 클라우드 인프라
  - 클라우드 엔지니어
  - 풀스택



팀원

- 이름 : 박장원
- 역할
  - 로봇 엔지니어
  - 클라우드 인프라
  - 클라우드 엔지니어



팀원

- 이름 : 한은선
- 역할
  - 데이터 분석
  - 클라우드 인프라
  - 클라우드 엔지니어



CONTENTS

목차

01 프로젝트 개요

02 프로젝트 계획 수립

03 개발 과정 및 현황

04 시연 영상

CONTENTS

01

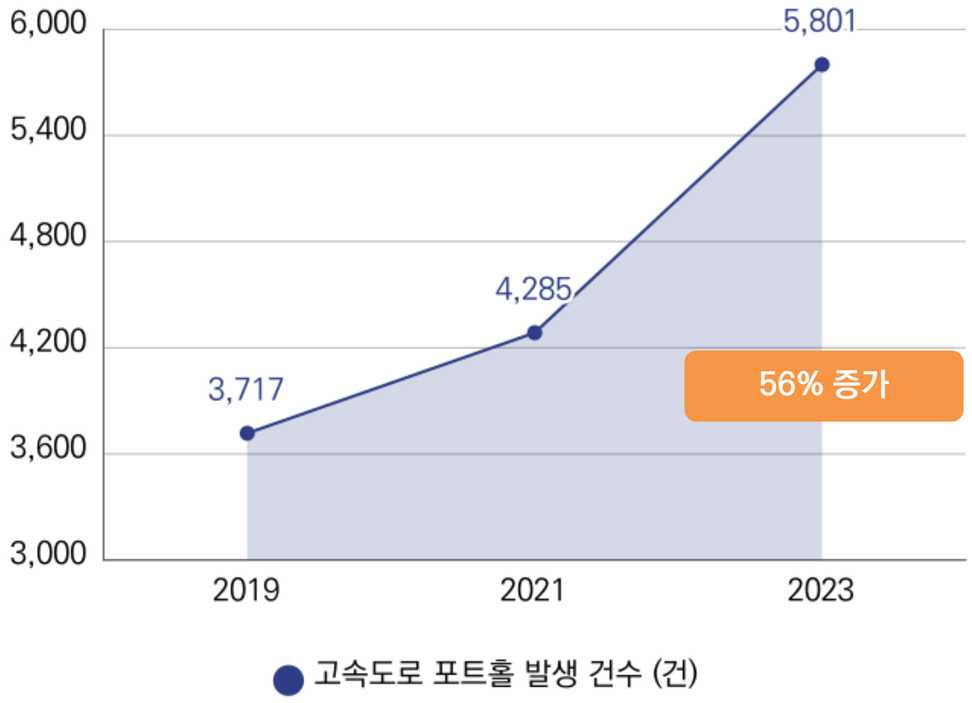
프로젝트 개요

같은 자리서 18대 '쿵' …
이상 기후에 '도로 위 지뢰' 포트홀 급증

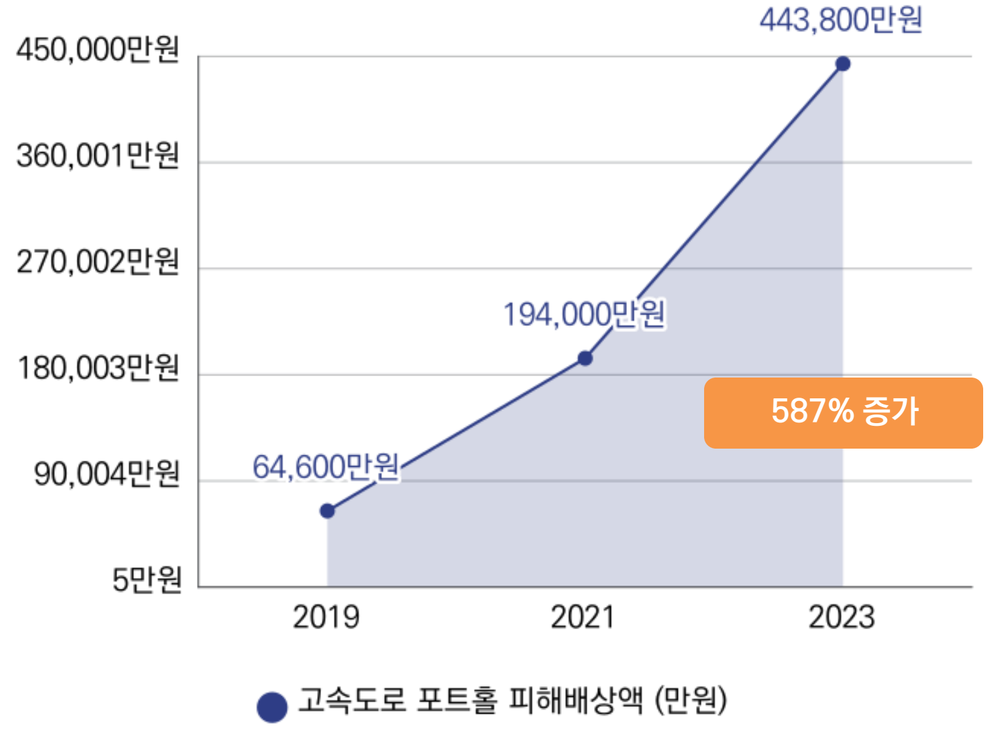


포트홀 사고 발생 및 피해 현황

최근 5년간 고속도로 포트홀 발생 건수 총 2만2752건



최근 5년간 포트홀 피해 배상액 총 119억3400만원



기존 포트홀 신고 서비스의 한계점

택시 포트홀 신고 시스템

서울시 '택시 포트홀 신고시스템' 최우수 협업사례 선정

한철 기자 | 입력 2014.12.15 13:38 | 댓글 0



도로이용불편신고 척척해결 서비스



도로이용불편
① 척척해결 서비스
어플을 실행한다



도로이용불편
② 신고촬영 버튼을
누른다

도로이용불편신고 척척해결서비스는
도로에서 발생하는 불편사항을
사진촬영 또는 신고전화 기능을 이용하여
해당 도로관리청에 실시간으로 신고하고,
처리 후 결과를
앱으로 전송 받을 수 있습니다

운전자가 운전 중에 도로 파손 발견 시 **직접 기기를 조작하는 신고 방식**과
신고 후 확인 절차가 이루어지는 **번거로움이 존재**

객체 탐지 기반 자율주행 차량 로봇을 활용한

포트홀 감지 및 자동 신고 서비스



포트홀 탐지 결과를 **실시간**으로 웹 서버에 전송하여
관련 기관이 신속하게 대응할 수 있는 시스템 구축이 목표

서비스 차별화 전략

기능 1. 실시간 객체 탐지

실시간 동적 객체 탐지

↓

제어 코드 전송

↓

상황에 따른 주행

기능 2. 자동 신고 시스템

실시간 동적 객체 탐지

↓

도로 정보 및 포트홀 정보 전송

↓

데이터베이스 및 웹페이지 업데이트

객체 탐지 기반 자율주행 차량 로봇을 활용한

포트홀 감지 및 자동 신고 서비스

기대 효과

- 신고 시스템 자동화를 통한 인력 소모 감소 및 효율적인 자원 관리
- 포트홀 발견 초기 신속한 대응으로 차량 파손 및 교통사고 예방
- 전반적인 교통 환경 개선 및 시민의 도로 이용 안전 강화
- 관련 기관의 효율적인 대응 지원



CONTENTS

02

프로젝트 계획 수립

2. 프로젝트 계획 수립

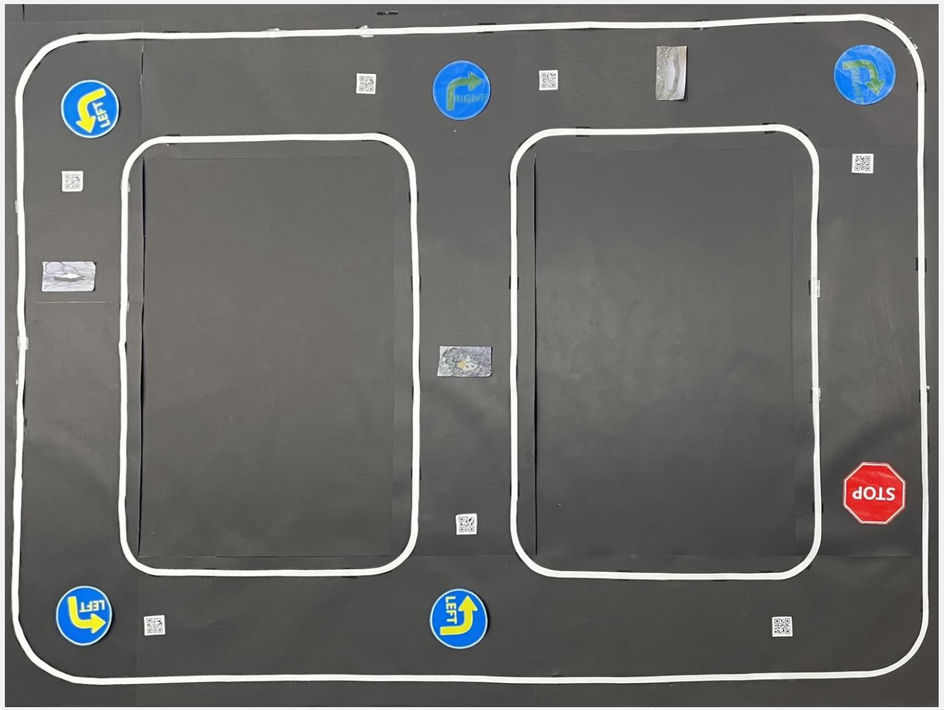
(1) 추진 일정

WBS

구분		8월				9월				10월				11월				12월			
		1W	2W	3W	4W	1W	2W	3W	4W	1W	2W	3W	4W	1W	2W	3W	4W	1W	2W	3W	4W
개발 기획	프로젝트 목표 설정																				
	기획 설계																				
	계획 수립																				
개발 준비	시스템 파이프라인 구성 기획																				
	인프라 구성 기획																				
	모델 구성 기획																				
개발 진행	모델 학습 및 최적화																				
	제어 로직 개발																				
	스트리밍 환경 구축																				
	프로세스 통합																				

주행 환경 구성

트랙 구성



구성 요소	세부 사항
도로 유형	1차선 일반도로
도로 폭	30cm
위치 데이터	QR코드 스캔

주행 환경 구성

선정 근거

✓ 실제 도로 환경에서 흔히 볼 수 있는 형태의 도로 유형으로 실제 도로 상황과 유사한 조건에서 테스트 가능

✓ OpenCV 라이브러리를 통해 추출한 차선 및 중앙값을 제어에 반영하기 위해 노이즈를 최소화할 수 있는 실선의 1차선 일반 도로를 구성

구성 요소	세부 사항
도로 유형	**1차선 일반도로**
도로 폭	30cm
위치 데이터	QR코드 스캔

주행 환경 구성

선정 근거

✓ 국토교통부령 도로구조규칙에 따르면 도로 구분에 관계없이 최소 도로폭은 3.00m 이상,
설계기준자동차의 승용자동차의 폭은 1.7m

✓ 실제 도로 환경과 유사한 주행 환경을 구성하기 위해 로봇 차량의 폭이 17cm임을 고려하여 실제 도로 환경을 10배 축소하여 도로 폭을 30cm로 구성

구성 요소	세부 사항
도로 유형	1차선 일반도로
도로 폭	**30cm**
위치 데이터	QR코드 스캔

주행 환경 구성

선정 근거

- ✓ ONLY 카메라 -> SLAM 기술 활용 불가
  실내 주행 -> GPS 수신 어려움
  환경적 제약과 비용 효율성 고려
- ✓ 좌표값 대신 도로 번호를 담은 QR코드 활용
- ✓ 실내 주행 로봇의 위치 정보 수집 방식 중 하나인
  QR코드 스캔 방식 선정

구성 요소	세부 사항
도로 유형	1차선 일반도로
도로 폭	30cm
위치 데이터	**QR코드 스캔**

주행 조건 설정

구성 요소	세부 사항
정지	로봇이 객체 탐지 시 감속 후 정지
우회전/좌회전	로봇이 객체 탐지 시 회전 후 정상 주행
포트홀	로봇이 객체 탐지 시 감속 후 정상 주행
차선	로봇이 차선 검출 후 중앙값을 추적하여 주행

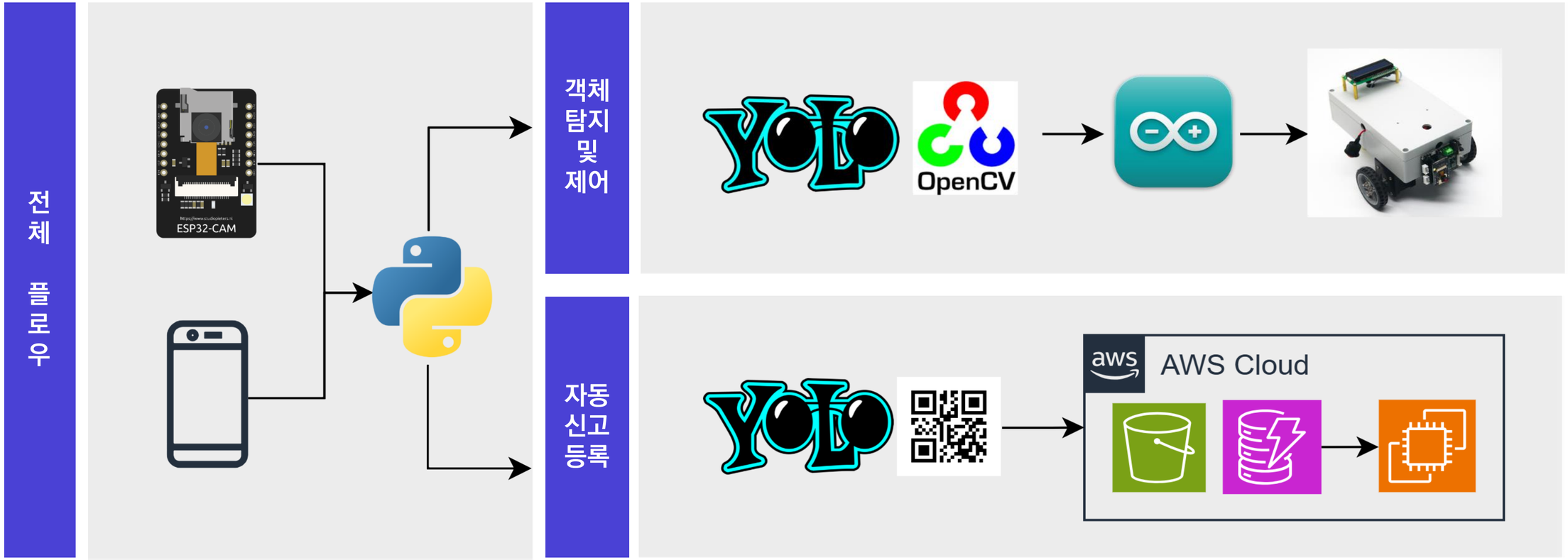
CONTENTS

03

개발 과정 및 현황

- 시스템 설계

시스템 플로우



개발 과정 및 현황

(1)

소프트웨어 구성

객체 탐지 기술 선정

진동 기반 탐지	영상 인식 기반 탐지	레이저 계측 기반 탐지
차량이 겪는 진동을 계측하여 노면 파손이 유발한 진동 추출	카메라를 통해 수집한 이미지를 분석 후 객체 탐지	레이저 빛에 반사된 신호를 분석하여 노면 상태를 평가할 지표 생산
✓ 저렴한 비용 ✓ 실시간 평가에 적합 ✓ 차량 바퀴의 주행 경로 외의 노면 손상 탐지 불가	✓ 저렴한 비용 ✓ 실시간 평가에 적합 ✓ 기술 도입 용이	✓ 노면 상태 정밀 평가 가능 ✓ 높은 비용 ✓ 기술 도입 어려움 ✓ 다량의 포트홀 신속하게 탐지 어려움

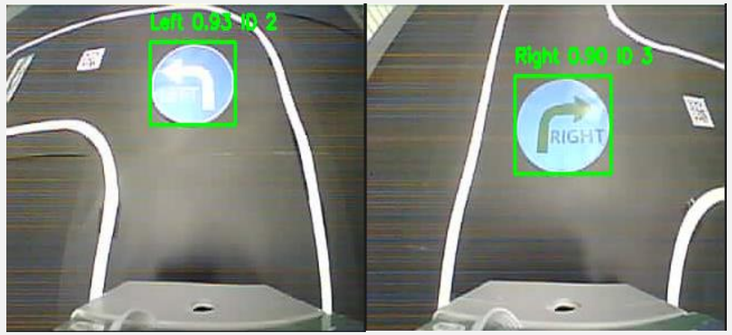
도로 노면의 포트홀을 **실시간**으로 인식하고 신고하는 시스템에
가장 적합한 **영상 인식 기반 객체 탐지 방식**을 채택

객체 탐지 기술 선정

영상 인식 기반 탐지

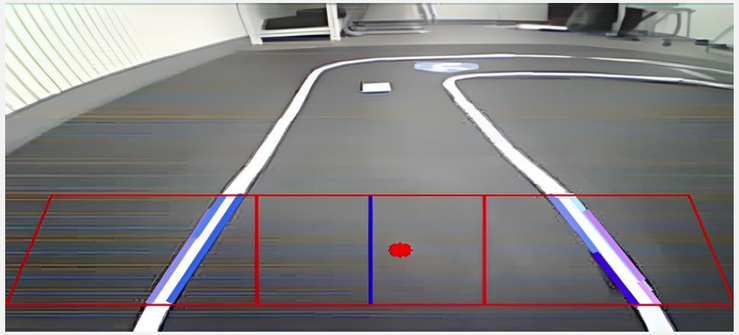
YOLO

객체 탐지 결과에 따른 **주행 제어** 적용



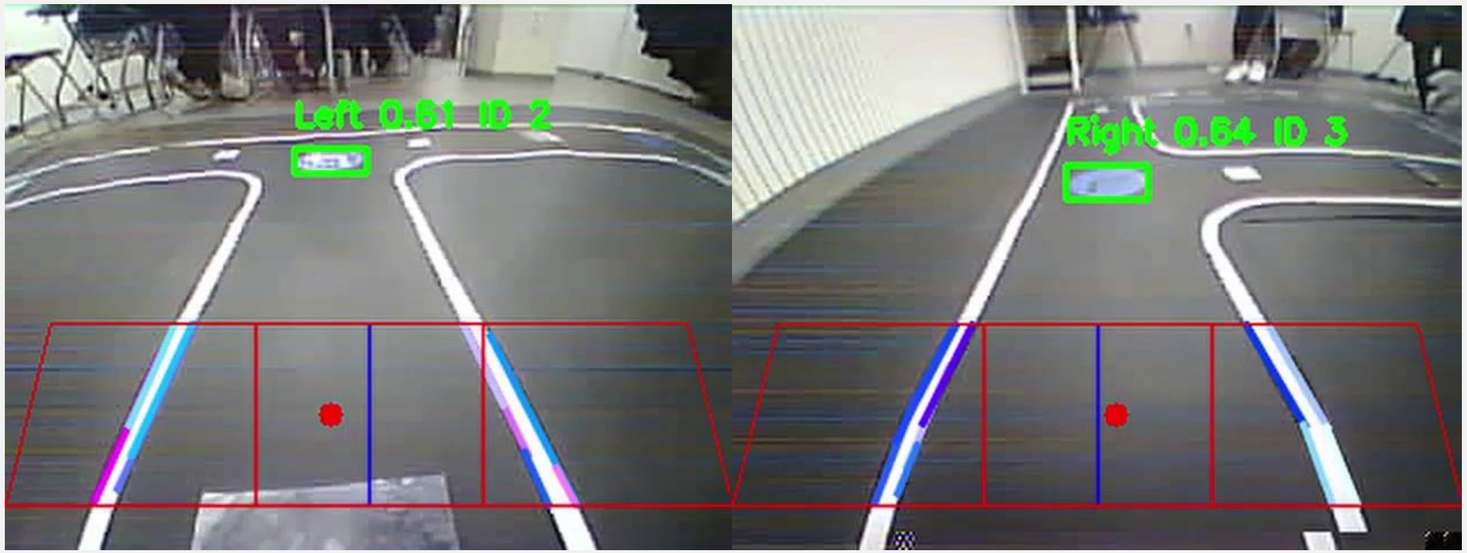
OpenCV

차선 검출 결과에 따른 **중심점 제어** 적용



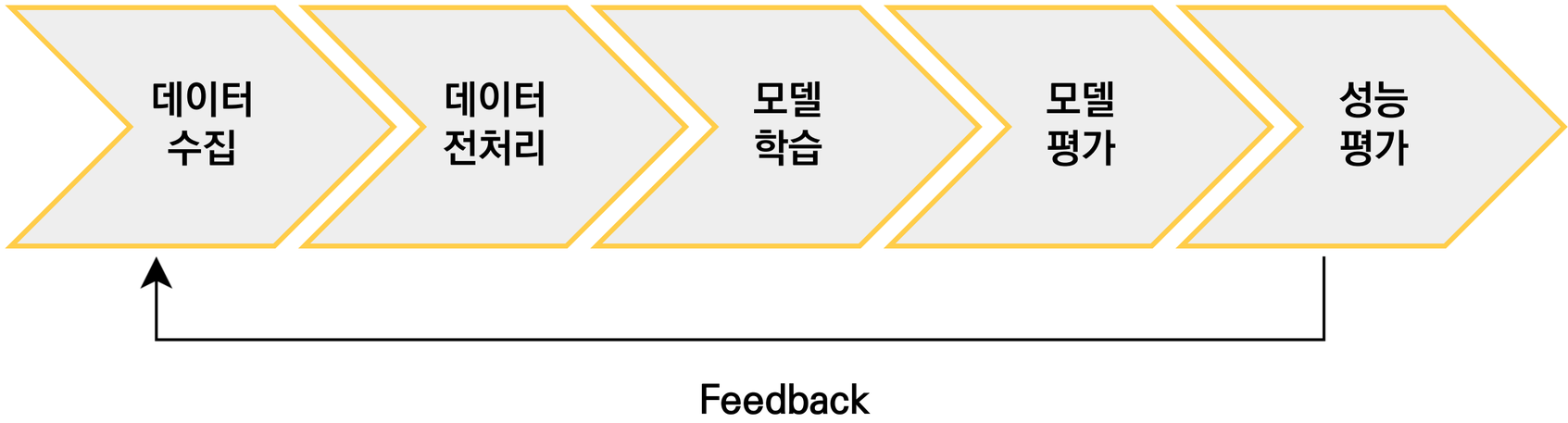
객체 탐지 기술 선정

영상 인식 기반 탐지



YOLO와 OpenCV를 활용한 객체 인식 및 차선 검출 결과를 바탕으로
주행 중 **차량의 중앙점을 검출하고 차량 제어를 수행**

YOLO 프로세스 흐름도

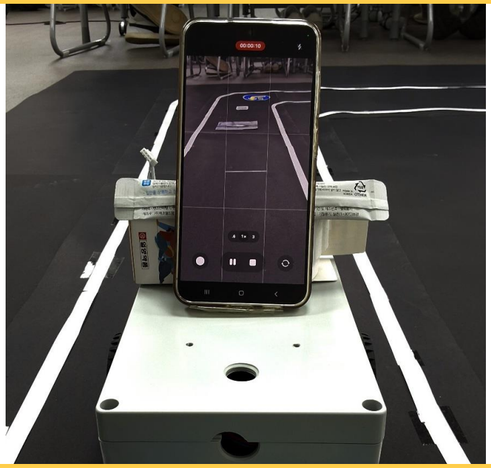


- 모델 학습 과정

YOLO 학습 과정

데이터 수집 → 데이터 전처리 → 모델 학습 → 모델 평가 → 성능 평가

실제 주행 환경에서 촬영한 영상



자체 데이터셋 구축



실제 주행 환경에서 촬영한 동영상 데이터 수집

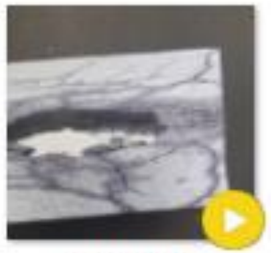
- 모델 학습 과정

YOLO 학습 과정

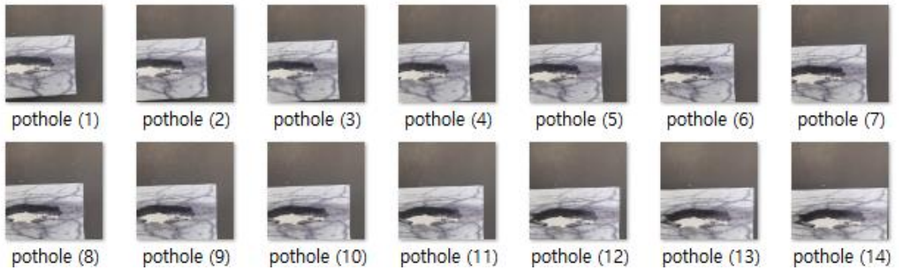
데이터 수집 → **데이터 전처리** → 모델 학습 → 모델 평가 → 성능 평가

(1) 이미지 데이터 생성

영상 데이터



이미지 데이터



초당 60프레임으로 나누어 이미지 데이터 셋 구축

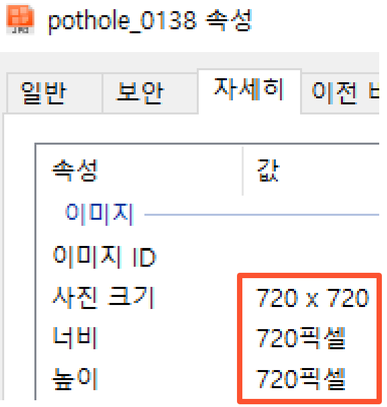
- 모델 학습 과정

YOLO 학습 과정

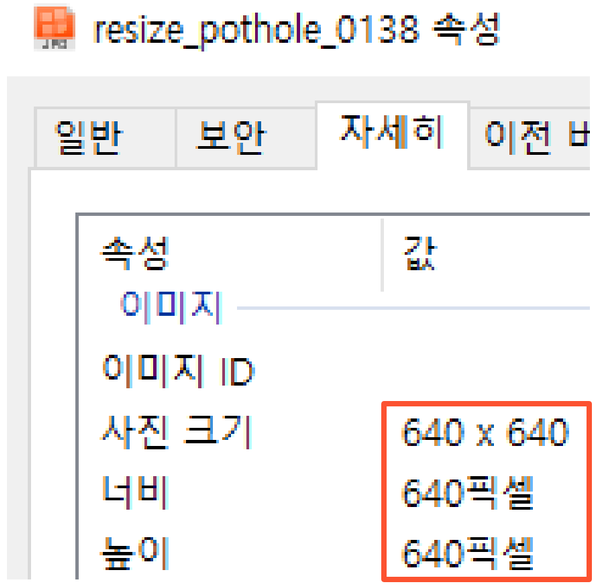
데이터 수집 → **데이터 전처리** → 모델 학습 → 모델 평가 → 성능 평가

(2) 데이터 리사이징

다양한 크기의 데이터



정규화된 크기의 데이터



이미지 크기를 640 x 640 픽셀 크기로 맞추어 저장

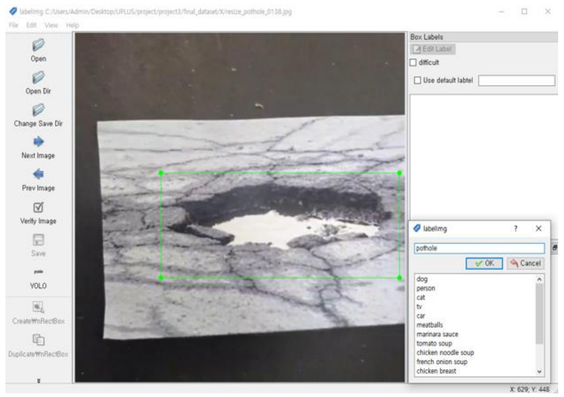
- 모델 학습 과정

YOLO 학습 과정

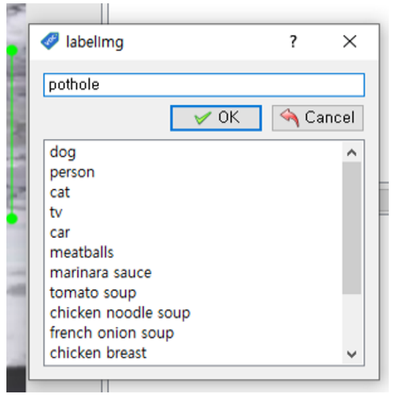
데이터 수집 → **데이터 전처리** → 모델 학습 → 모델 평가 → 성능 평가

(3) 데이터 라벨링

바운딩 박스 영역 지정



클래스 지정



바운딩 박스 영역 표시 후 클래스 지정하여 학습용 데이터 생성

- 모델 학습 과정

YOLO 학습 과정

데이터 수집 → 데이터 전처리 → **모델 학습** → 모델 평가 → 성능 평가

모델 학습

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

results = model.train(data="C:/Users/Admin/Desktop/UPLUS/project/project3/final_dataset/data.yaml", epochs=80, imgsz=640)
```

학습 횟수

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size

               Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 6/6 [00:00<0
                 all        192        192      0.763      0.761      0.799      0.616

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size

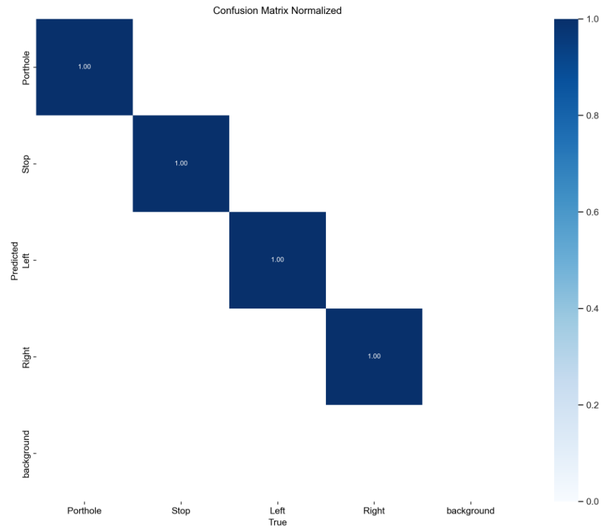
               Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 6/6 [00:00<0
                 all        192        192      0.871      0.974      0.971      0.729
```

- 모델 학습 과정

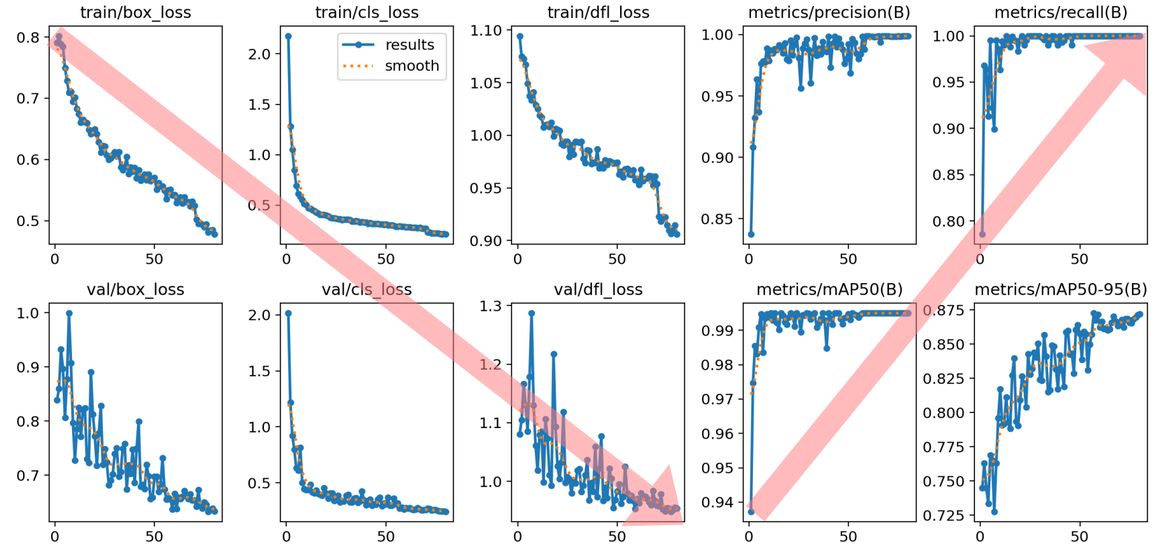
YOLO 학습 과정

데이터 수집 → 데이터 전처리 → 모델 학습 → **모델 평가** → 성능 평가

confusion matrix



손실률 & 정확도



클래스별 정확도

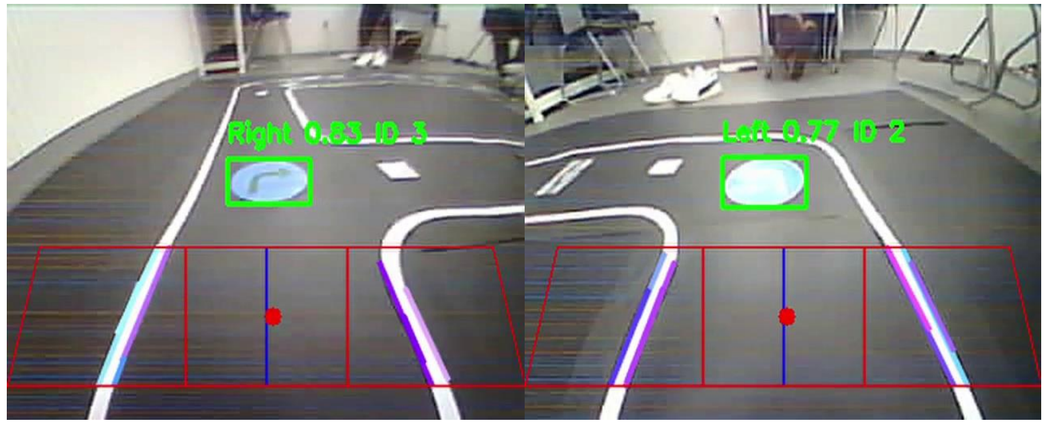
Class	Images	Instances	Box(P	R	mAP50	mAP50-95):
all	192	192	0.998	1	0.995	0.86
Pothole	53	53	0.998	1	0.995	0.889
Stop	41	41	0.999	1	0.995	0.906
Left	47	47	0.998	1	0.995	0.828
Right	51	51	0.997	1	0.995	0.817

- 모델 학습 과정

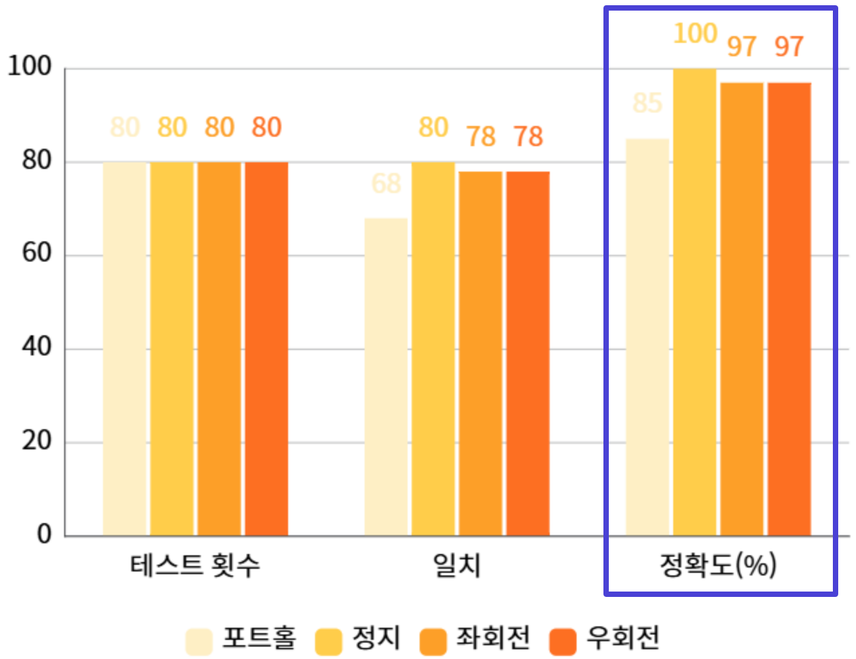
YOLO 학습 과정

데이터 수집 → 데이터 전처리 → 모델 학습 → 모델 평가 → **성능 평가**

모델 테스트 주행



클래스별 성능 평가 결과



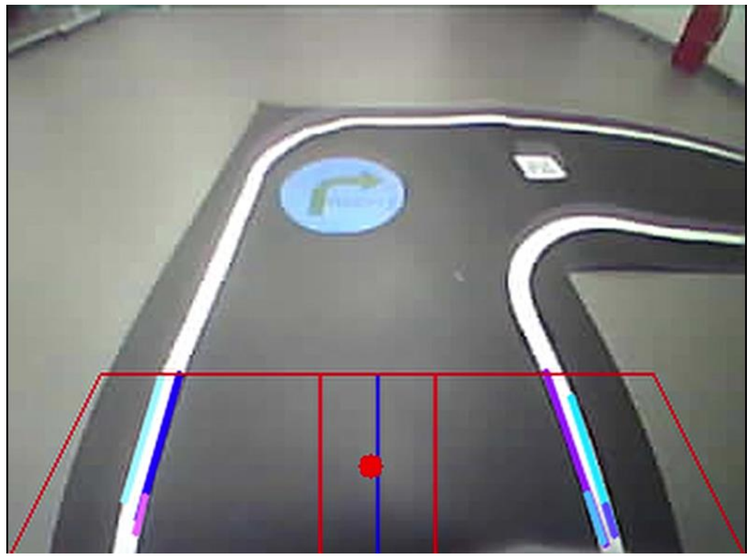
성능 평가 결과는 모델 평가보다 다소 낮은 정확도를 보이지만
주행 환경에 영향을 최소화하면서 높은 정확도를 유지함

- 차선 검출

OpenCV 프로세스 흐름도

ROI 설정	이미지 전처리	엣지 검출	직선 검출	차선 검출 및 제어
이미지 처리에서 특정 관심 영역 설정	흑백 이미지 변환 및 노이즈 제거	색상 및 밝기 차이를 통해 경계 검출	경계를 기준으로 직선 검출	차선 검출 후 중앙 계산 및 방향 제어

“ 메모리 사용량 감소
처리 속도 향상 ”

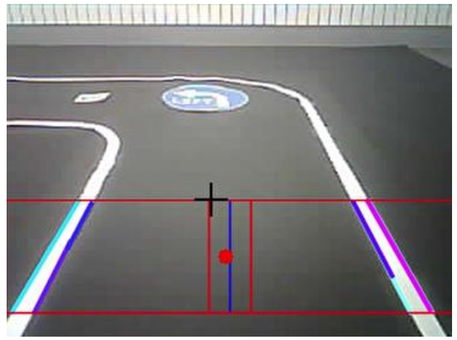


- 차선 검출

OpenCV 프로세스 플로우

ROI 설정	이미지 전처리	엣지 검출	직선 검출	차선 검출 및 제어
이미지 처리에서 특정 관심 영역 설정	흑백 이미지 변환 및 노이즈 제거	색상 및 밝기 차이를 통해 경계 검출	경계를 기준으로 직선 검출	차선 검출 후 중앙 계산 및 방향 제어

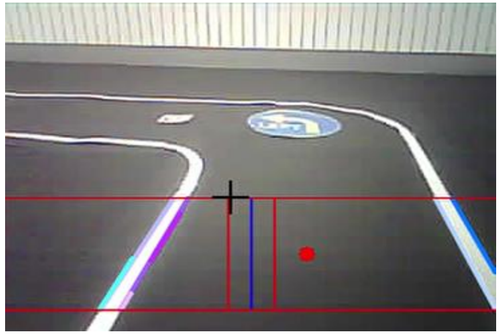
중앙 제어



```
0: 480x640 (no detections), 44.0ms
Speed: 1.0ms preprocess, 44.0ms inference, 0.0ms postprocess per image at shape (1,
3, 480, 640)
50 50
```

속도 (50,50)

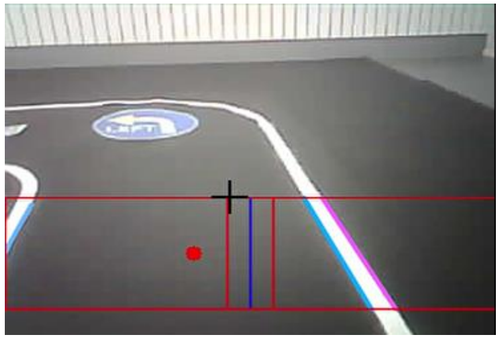
오른쪽 제어



```
0: 480x640 (no detections), 37.0ms
Speed: 1.0ms preprocess, 37.0ms inference, 0.0ms postprocess per image at shape (1,
3, 480, 640)
30 20
```

속도 (30,20)

왼쪽 제어



```
0: 480x640 (no detections), 33.0ms
Speed: 1.0ms preprocess, 33.0ms inference, 0.0ms postprocess per image at shape (1,
3, 480, 640)
20 30
```

속도 (20,30)

(2)

하드웨어 구성

로봇 차량 선정



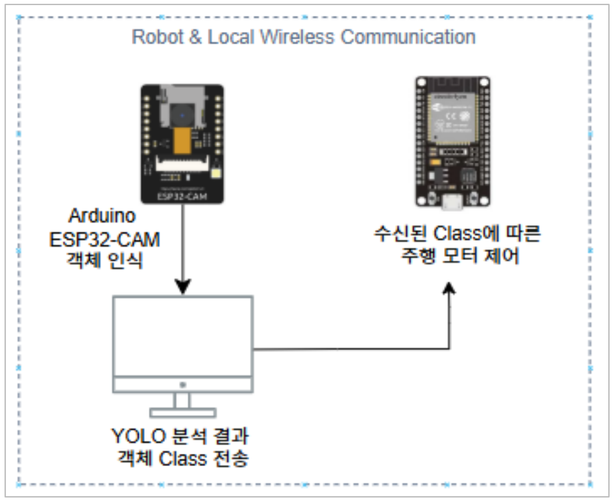
로봇 선정 POINT

- 2륜 구동 차량
- ESP32-cam 사용
- Step-motor 사용

- 통신 방식 선정

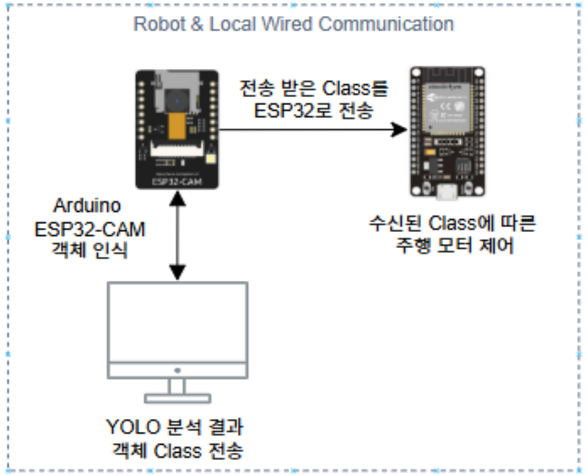
제어 신호 통신

ESP32로 직접 무선 통신



VS

ESP32-CAM과 ESP32의 유선 통신



WI-FI 통신 사용을 줄여 **시스템 부하 감소**
ESP32-CAM과 ESP32의 **유선 통신이 더 빠르고 안정적**

영상 스트리밍 통신

URL 기반 통신	VS	TCP 통신
클라이언트가 서버에 요청을 보내고 서버가 응답을 반환하는 구조		클라이언트와 서버가 소켓을 통해 연결된 상태에서 지속적으로 데이터를 송수신하는 구조
✓ 단순 요청-응답 형식 ✓ 불연속적 통신 ✓ 높은 지연 시간		✓ 실시간 처리 ✓ 빠른 데이터 전송 ✓ 데이터 스트리밍 최적화

TCP 통신은 **실시간** 데이터 전송과 **지속적인 양방향 통신**이 필요한 상황에서 더 빠르고 안정적인 성능을 제공하며, 데이터 **전송 속도**와 시스템 부하를 **최적화**

개발 과정 및 현황

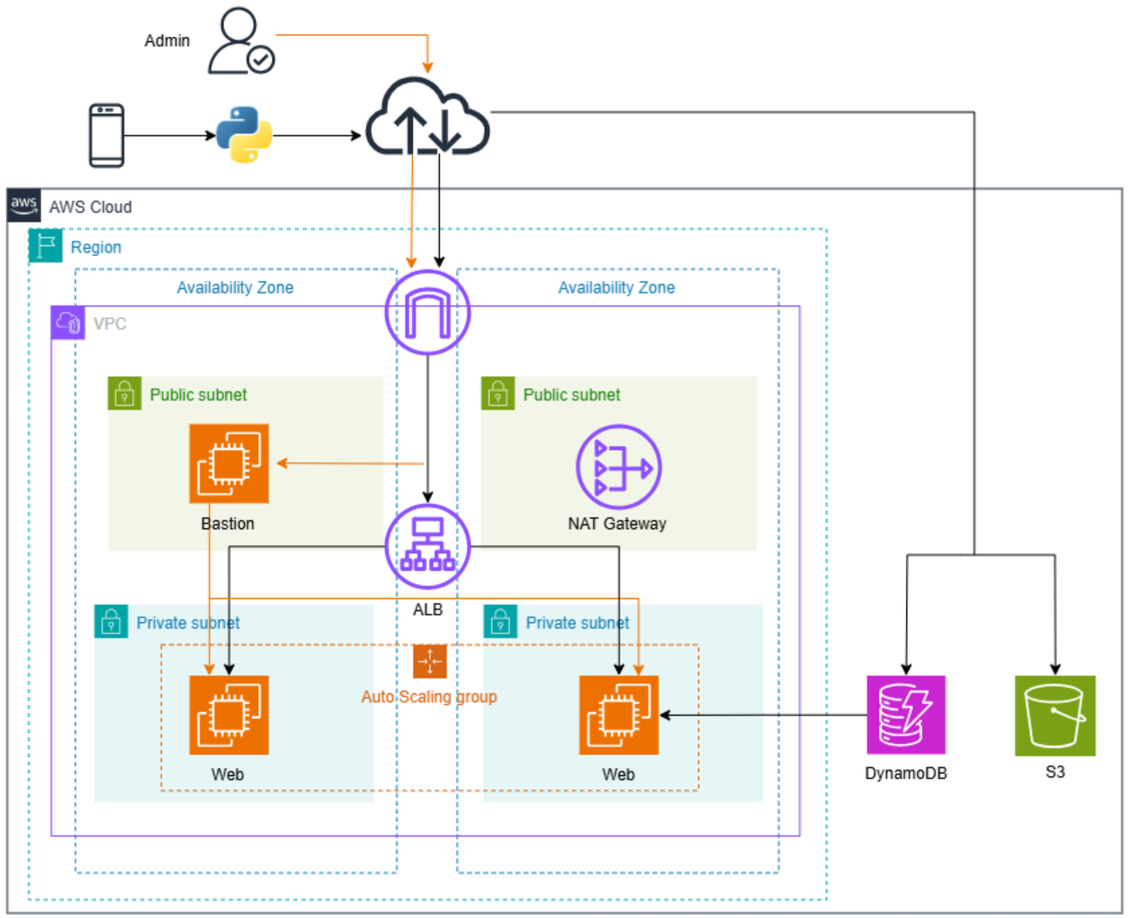
(3)

클라우드 인프라 구성

- 인프라 구성

AWS 인프라 아키텍처

AWS



핵심 파이프라인 (1)

스마트폰 CAM





도로 스캔



도로 정보 인식



포트홀 및 도로 정보 업데이트



객체 탐지



포트홀 이미지 업데이트



핵심 파이프라인 (2)

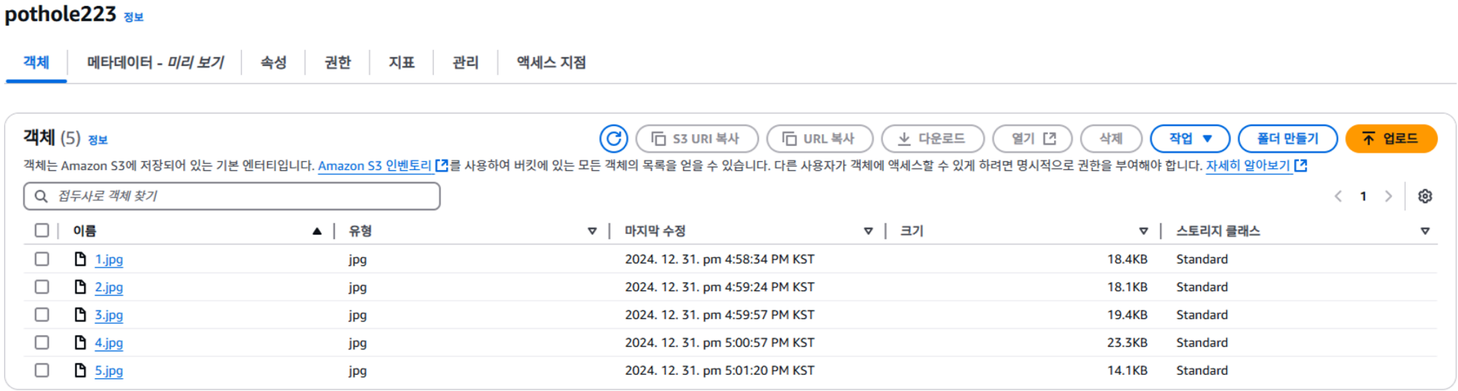
포트홀 및 도로 정보 업데이트



포트홀 이미지 업데이트



S3 업데이트



핵심 파이프라인 (2)

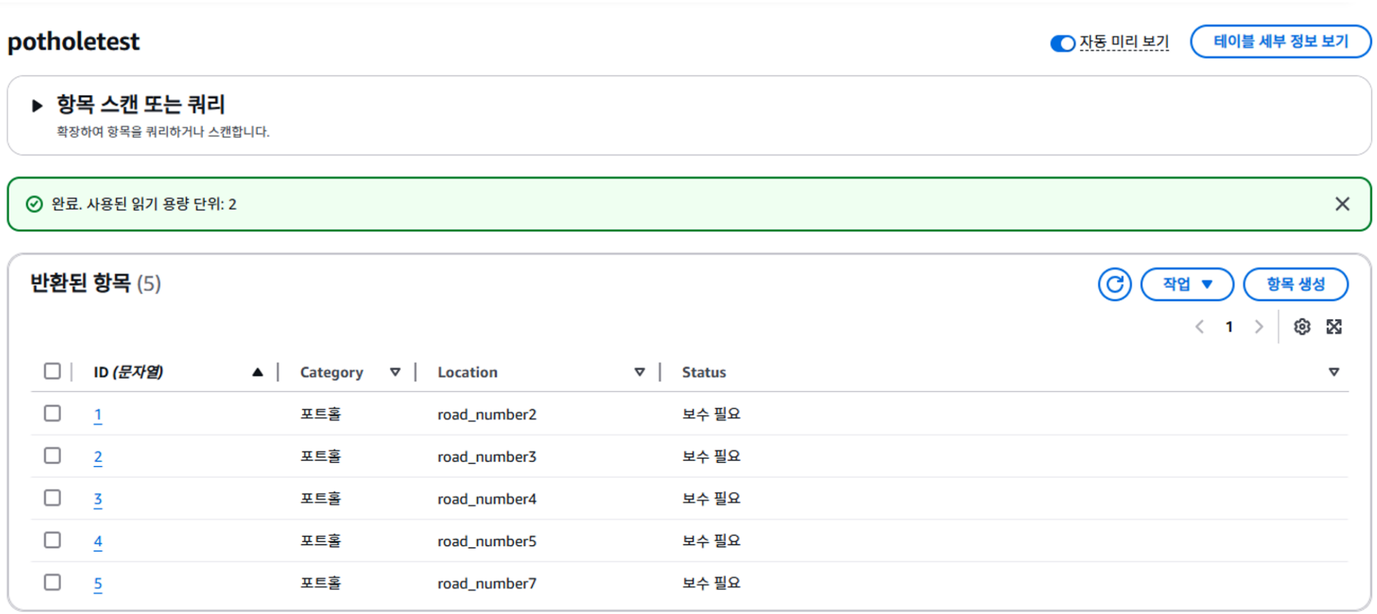
포트홀 및 도로 정보 업데이트



포트홀 이미지 업데이트



Dynamo DB 업데이트



핵심 파이프라인 (2)

포트홀 및 도로 정보 업데이트



포트홀 이미지 업데이트



웹페이지 업데이트



CONTENTS

04

시연 영상

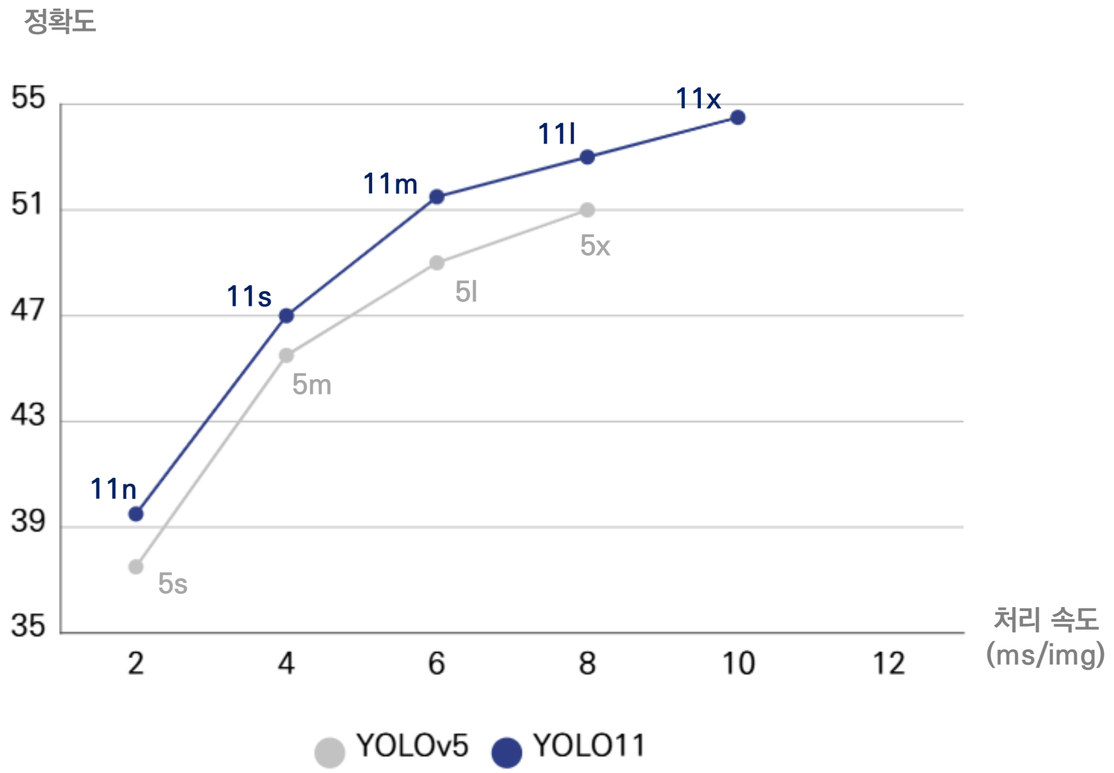
감사합니다 !



Q&A

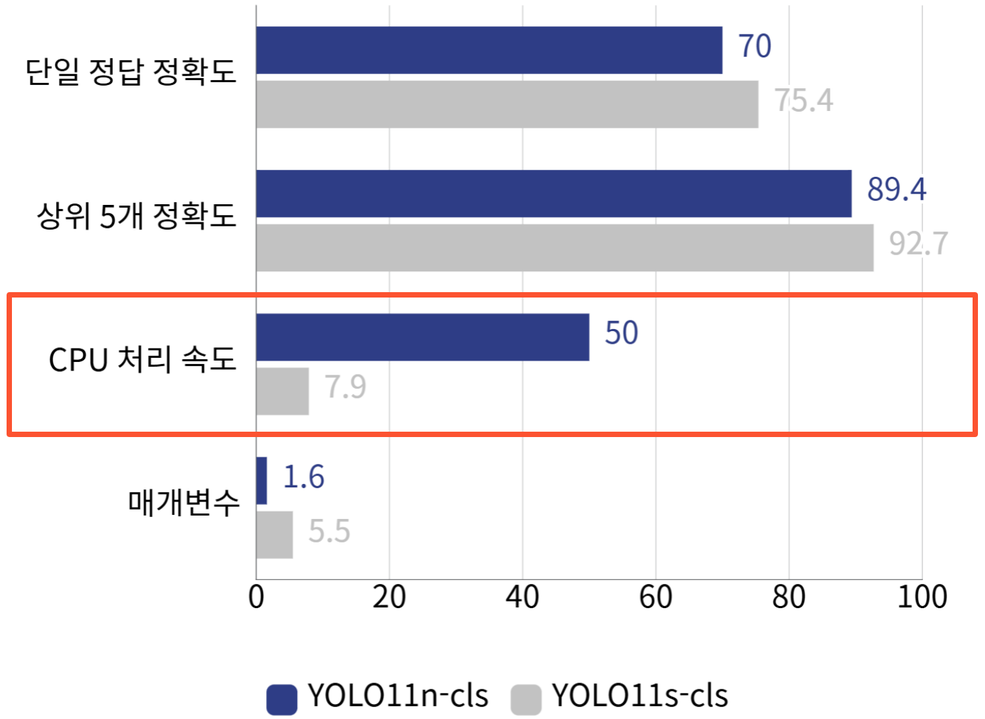


YOLO 모델 선정



빠른 처리 속도와 정확도로 인해 실시간 객체 탐지에 일반적으로 사용되는 YOLOv5와 YOLOv11 비교 분석
처리 속도는 두 모델 모두 우수하나 **정확도가 더 높은 YOLOv11 모델 선정**

YOLO 모델 선정



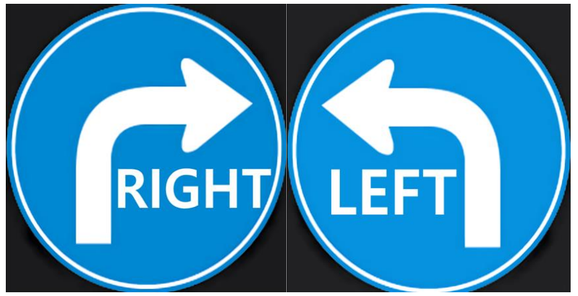
데이터 개수가 비교적 적은 본 프로젝트에서는 두 모델 모두 정확도는 0.9로 동일
따라서 **실시간 처리와 카메라 통신에 유리한 YOLO11n-cls 사용**

데이터 수집

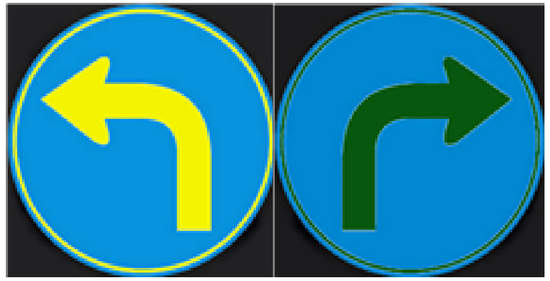
DATA SET 1
공공 데이터셋



DATA SET 2
자체 데이터셋 : 영문



DATA SET 3
자체 데이터셋 : 색상

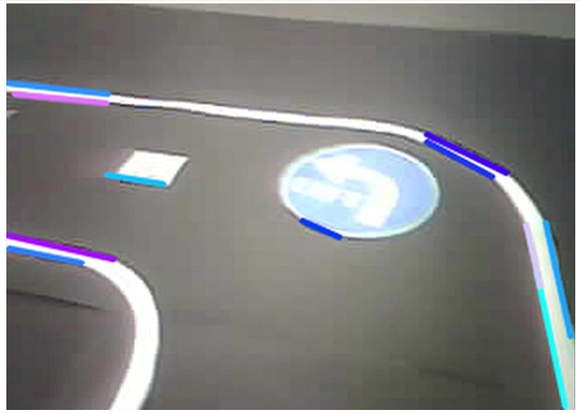


DATA SET 4
자체 데이터셋 : 색상 + 영문

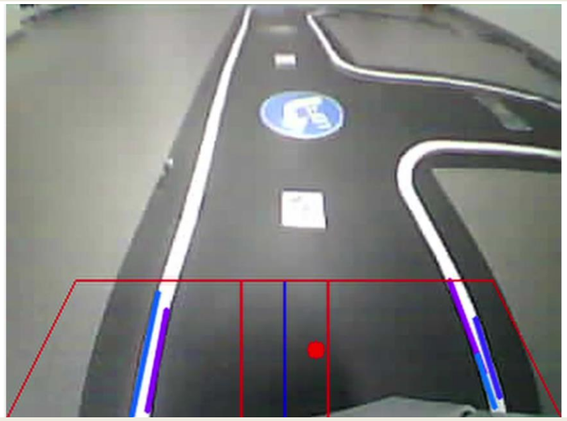


차선 인식

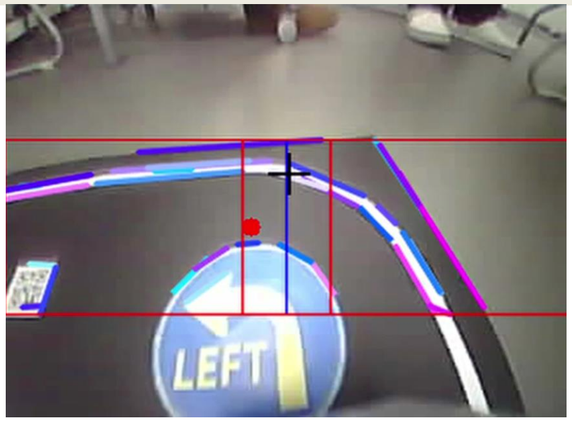
1. OpenCV 차선 인식 시도



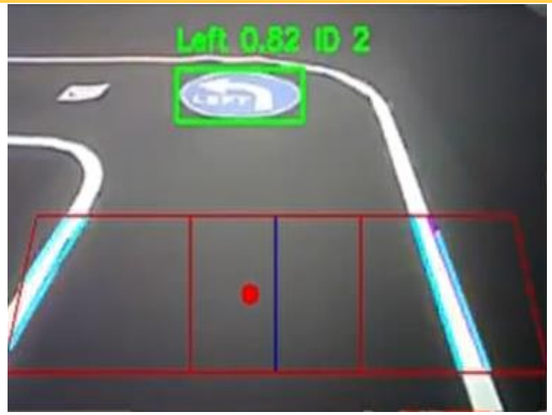
2. ROI로 인식 범위 제한



3. 색상 값 추출



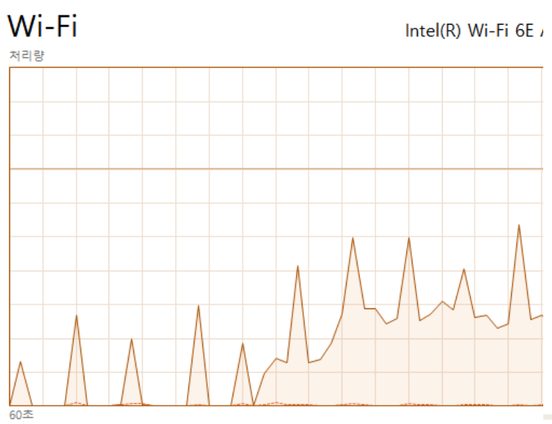
4. 도로 확장



부록

딜레이 해결

1. 와이파이 -> 핫스팟



2. 두 개의 캠 사용

ESP32-CAM
로봇 제어

PHONE-CAM
포트홀 신고

3. 해상도

640 X 480
(VGA)

320 X 240
(QVGA)

4. 프레임

일반
스트리밍
(FPS60)

프레임 스킵
(FPS10)

5. 통신 방법 변경

URL
스트리밍

TCP
소켓 통신

도로 정보 저장

객체 중복 저장 방지

동일한 객체를 여러 번 인식하여
데이터가 중복으로 업로드되는 문제 발생

✓ 바운딩 박스 중심 좌표가 프레임 중앙 영역 (ROI)에 있을 경우에만 업로드

✓ 프레임의 가운데 40%영역만 ROI로 설정

✓ 중심 좌표가 ROI의 범위 안에 있는 경우 True 반환

도로 중복 저장 방지

각 도로 정보에 따른
포트홀 신고 내역이 중복 저장되는 문제 발생

✓ QR 코드를 통해 각 도로를 고유하게 식별

✓ 이미 처리된 QR코드 저장 방지

✓ 도로 정보가 중복되지 않을 때만 업로드